BSc (Hons) in **Information Technology**
**Specialized in AI**

# IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 04

**Objective & Lecture Mapping:** In previous labs, you explored Uninformed Search strategies like BFS and UCS inside `agent.py` [cite: 1]. Today, we transition to **Informed Search**. You will enhance your agent by implementing the **A* Search** algorithm, using **Heuristics** to drastically reduce the number of explored nodes in the `visual_grid_game.py` environment[cite: 4]. You will start with basic heuristic functions and connect them to your search logic.

## Part 1: Practical Implementation — Building the A* Agent

### Step 1.1: Implementing the Heuristic Functions

Informed search algorithms require a heuristic function, h(n), which estimates the cheapest path from the current node to the goal. You will calculate distance metrics on a 2D grid.

1. Create a new Branch called Week_04 in your Forked Github Repo.
2. Open `agent.py` and locate your `SearchAgent` class[cite: 1].
3. Add a new method named `def manhattan_distance(self, pos, goal):` that calculates the distance using the formula h(n) = |x_1 - x_2| + |y_1 - y_2| and returns the integer value.
4. Add a second method named `def euclidean_distance(self, pos, goal):` that calculates the straight-line distance using the formula h(n) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}. You will need to `import math` for this.
5. *Testing Checkpoint:* Print the outputs of both functions for a mock start position (0, 0) and goal (3, 4) to verify that Manhattan returns 7 and Euclidean returns 5.0.

### Step 1.2: Implementing A* Search

> A* evaluates nodes by combining the path cost so far, g(n), and the estimated cost to the goal, h(n). The evaluation function is f(n) = g(n) + h(n).

1. Inside `SearchAgent`, create a new method: `def astar_search(self, start_pos, goal_pos, walls, grid_size, heuristic_type='manhattan')`.
2. Initialize an empty priority queue using `heapq` and an empty set for `reached_states`.
3. Push the initial state into the priority queue. Unlike UCS which only uses g(n), your A* tuple must be formatted as: `(f_cost, g_cost, current_pos, path_taken)`. For the starting node, g(n) = 0. Calculate h(n) using your chosen heuristic method, and set f(n) = g(n) + h(n).
4. Create your standard `while` loop to process the queue. Pop the node with the lowest `f_cost`. If `current_pos == goal_pos`, return the `path_taken`. Otherwise, add `current_pos to reached_states`.
5. For node expansion, check the four adjacent cells (Up, Down, Left, Right). For each valid neighbor (not a wall, within bounds, and not reached): Calculate g_{new} = g_{current} + 1, h_{new} using your heuristic, and f_{new} = g_{new} + h_{new}. Push the new tuple to the priority queue.

### Step 1.3: Integrating A* into the Agent's Decision Loop

> Finally, you need to connect your new search algorithm to the agent's perception and action loop so it can run in the visual environment.

1. Navigate to the `sense_and_act(self, percept)` method in your `SearchAgent`.
2. Add an `elif self.active_algo == 'AStar':` block to handle the new algorithm alongside your existing BFS/DFS/UCS.
3. Extract the necessary global state from the percept dictionary (e.g., `percept['remaining_food']`).
4. Find the closest food item to act as the `goal_pos`. Call your `astar_search` method and store the returned list of actions in `self.plan`.

5. Open `visual_grid_game.py` and modify the `GridGameGUI` initialization to inject your `SearchAgent()` instead of the `ModelBasedAgent()`. Run the simulation and observe the optimized pathfinding!

## Part 2: Theoretical Evaluation

> Based on your implementation and Lecture 05, complete the following analytical questions.

**1. (Understand) What is the key difference between how Uniform-Cost Search (UCS) and A* Search prioritize which node to explore next?**

The main difference between UCS and A* is the information they use to decide which node should be explored next. UCS prioritizes nodes only according to the actual cost already travelled from the starting state, represented by **g(n)**. It therefore chooses the node with the lowest accumulated path cost. A* also considers the cost already travelled, but it adds a heuristic estimate of the remaining distance to the goal. Its evaluation function is **f(n) = g(n) + h(n)**. Therefore, UCS is guided only by the cost from the start, while A* is guided by both the cost already travelled and an estimate of the cost remaining to reach the goal. This heuristic allows A* to focus its search toward the goal more efficiently.

**2. (Analyze) In Step 1.1, you used Manhattan Distance. Why is Manhattan Distance considered an "admissible" heuristic for this specific 4-way movement grid, and what would happen to your A* algorithm if the heuristic was NOT admissible?**

The main difference between UCS and A* is the information they use to decide which node should be explored next. UCS prioritizes nodes only according to the actual cost already travelled from the starting state, represented by **g(n)**. It therefore chooses the node with the lowest accumulated path cost. A* also considers the cost already travelled, but it adds a heuristic estimate of the remaining distance to the goal. Its evaluation function is **f(n) = g(n) + h(n)**. Therefore, UCS is guided only by the cost from the start, while A* is guided by both the cost already travelled and an estimate of the cost remaining to reach the goal. This heuristic allows A* to focus its search toward the goal more efficiently.

**3. (Evaluate) If we modified `visual_grid_game.py` to allow the agent to move diagonally (8-way movement), would Manhattan distance still be an admissible heuristic? Why or why not? Which metric should you switch to?**

No, Manhattan Distance would generally not be an appropriate admissible heuristic for an 8-way movement grid when diagonal movement has the same movement cost as horizontal and vertical movement. Manhattan Distance counts horizontal and vertical movements separately, so it can overestimate the actual cost when the agent can travel diagonally. For example, moving from `(0,0)` to `(3,3)` would have a Manhattan distance of 6, while three diagonal moves could reach the goal if diagonal movement costs 1. Therefore, Manhattan Distance would overestimate the true cost and would not be admissible under that cost model. For an 8-way grid where diagonal and straight moves have equal cost, a **Chebyshev distance** heuristic would be more appropriate because it reflects the ability to move in either axis simultaneously.

**4. (Create) When targeting multiple food items simultaneously, calculating the distance to just the single closest food item is a weak heuristic. Propose (in text) a stronger heuristic for navigating the grid to eat ALL remaining food efficiently.**

A stronger heuristic for eating all remaining food should consider the cost of visiting **all** food items rather than considering only the closest food item. One possible approach is to calculate the distance from the current position to one food item and then estimate the additional distances required to connect the remaining food items. For example, the heuristic could use a minimum spanning tree over the remaining food positions and add the distance from the agent's current position to the closest food. This gives an estimate of the total cost needed to connect and visit all remaining food without requiring the agent to calculate the complete route during every search step. Such a heuristic is stronger because it considers the relationships between multiple food locations rather than focusing only on the nearest pellet.

Once Completed Get a PDF of the completed File and Push into the Week 04 branch in the Github Project

Finalize and Lock Lab 04 Documentation

**Incomplete: {filledCount}/{fields.length} questions answered. Please provide detailed responses for all questions.**

**FACULTY OF COMPUTING**